MITTUNIVERSITETET

Laboration: System administration 2

Laborant: Hussein Alshaikhshaker

Kurs: DT149G

Datum: 2025-11-26

Bootprocess

Runlevels

Först bootade min maskin in i runlevel 5. Denna level avser graphical.target, det är ett tillstånd för multi-user med nätverk och grafiskt användargränssnitt. Men jag kör på ubuntu server och inte har gränssnitt, vilket är märkligt nog för att min ubuntu ska starta in i runlevel 5. Jag vet inte exakt vad det beror på men kan ha varit något med installationen av systemet. Jag exekverade kommandot [systemctl set-default multi-user.target] för att sätta multiuser till default och sedan exekverade [systemctl isolate reboot.target] för att starta om systemet. Efter omstart av systemet bootade OSet in i önskade runlevel. [4] [14]

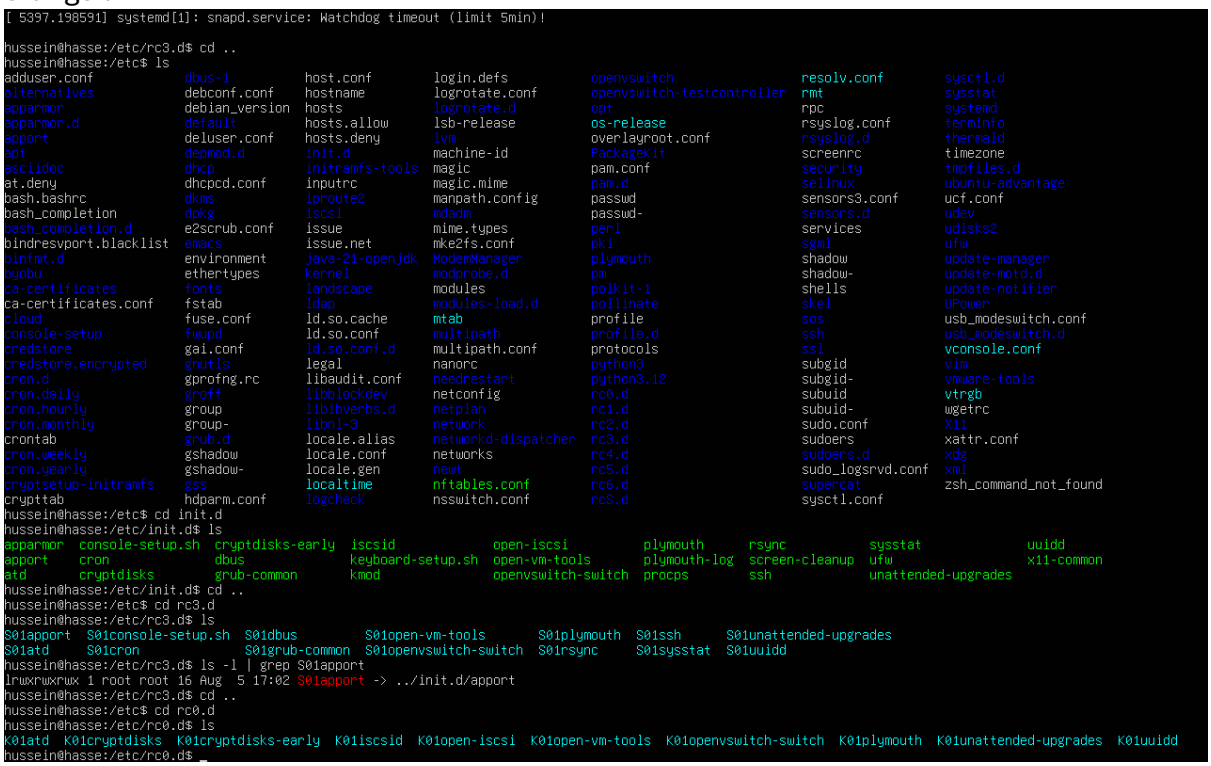


Figur 1: Systemets nuvarande runlevel

Länkarna /etc/rc*.d är symboliska länkar som pekar på de faktiska start- och stopp-skripten i /etc/init.d.
När runlevelvärdet ändras, exekveras alla länkar som finns i /etc/rc<runlevel>.d. När systemet stängs av, dvs runlevel ändras till 0, så körs init skripten som /etc/rc0.d pekar på.

I figur 2 ser man att jag har tagit apport skriptet som ett exempel. I /etc/rc3.d finns det en symbollänk som heter ”S01apport” och den pekar på /etc/init.d/apport.
S står för start, siffrorna som följer (i detta fall 01) styr ordningen i vilka skripts som ska köras först. Enligt figur 2 ser man att alla skripts inte ha konfigurerats till att köra i standard ordning som det bör vara i linux OS. Jag tror att det är för att ubuntu server är minimalt korrigerad. Det betyder inte att något fel kommer ske med exekveringen eller

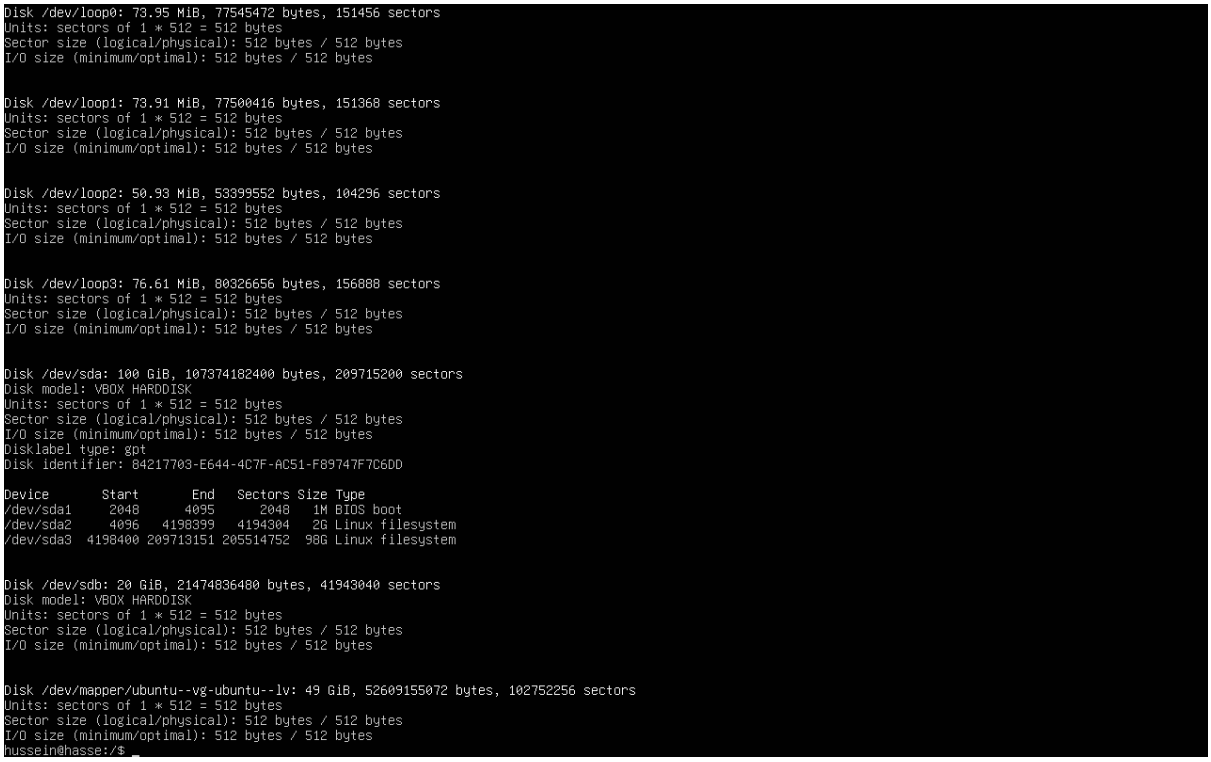
att skripten kommer att kunna köras samtidigt, init kommer att lägga dem i bokstavsordning. Jag har också listat filerna i RC0 där man ser att alla symbollänkar börjar med bokstaven K som står för ”Kill”, dessa avslutar processerna innan systemet stängs av.



Figur 2: Denna skärmdump visar init kataloger samt deras innehåll

The file system

Efter att jag skapade nya disken fick jag ut följande information med kommandot [fdisk -l]



Figur 3: Kommandot [fdisk -l]

Disk /dev/sdb är den nya disken, unmounted och oparitionerad.
Disken /dev/sda har delats i tre partitioner, sda1, sda2, sda3. Sda1 är för bootloader.

Flaggan för kommandot [df] för att visa resultat i bas 1024 är [-h]. Det står för human-readable. Jag skapade mappen /dump för mountpoint i rootkatalogen och mountade partiotionen /dev/sdb1 i den. Med kommandot [df -h] skrev jag ut ledigt utrymme i /dump, i bilden nedan ser man att jag använt 1% av hela partitionen och har 19GiB ledigt utrymme.

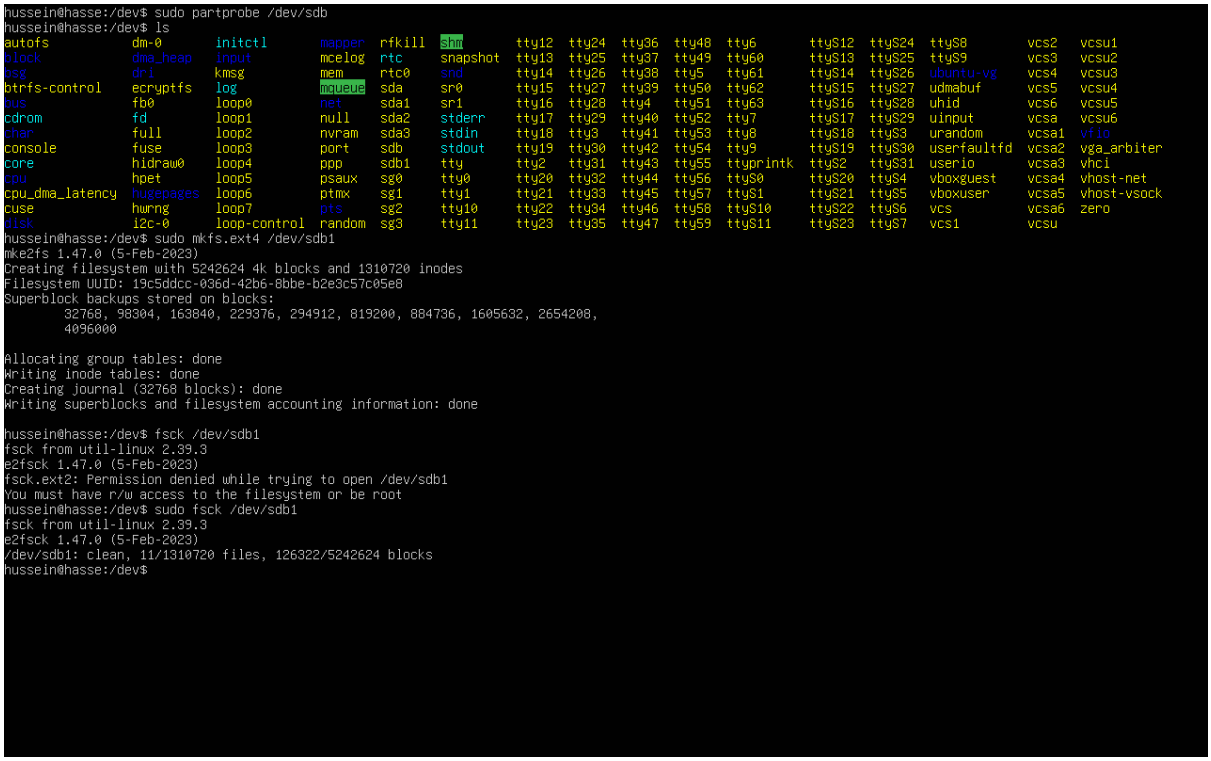


Figur 4: Kommandot [df -h]

Jag har ingen Desktop katalog då jag kör på ubuntu server men jag exekverade [du -sh ~], alltså min hemkatalog. Den tog upp 29GiB.

Jag har skapat en ny partition som täcker hela 20GiB. Programmet fdisk gav ut starndard värden för partitionsallokeringen.
Standard sektor storlek = 512B.
Min nya disk storlek = 20GiB. Total = 20 * 1024^3 = 21 474 836 480 bytes.
Antal sektorer = 21 474 836 480 / 512 = 41 943 040
Vissa sektorer i början av partitionen är reserverade och första sektorn är 0, dvs sista sektorn är 41 943 040 – 1.
Så jag satte första sektor till 2048 och sista 41 943 039, precis som programmet fdisk rekommenderade som default.

Sedan skapade jag ett ext4 filsystem med kommandot [sudo mkfs.ext4 /deb/sdb1].

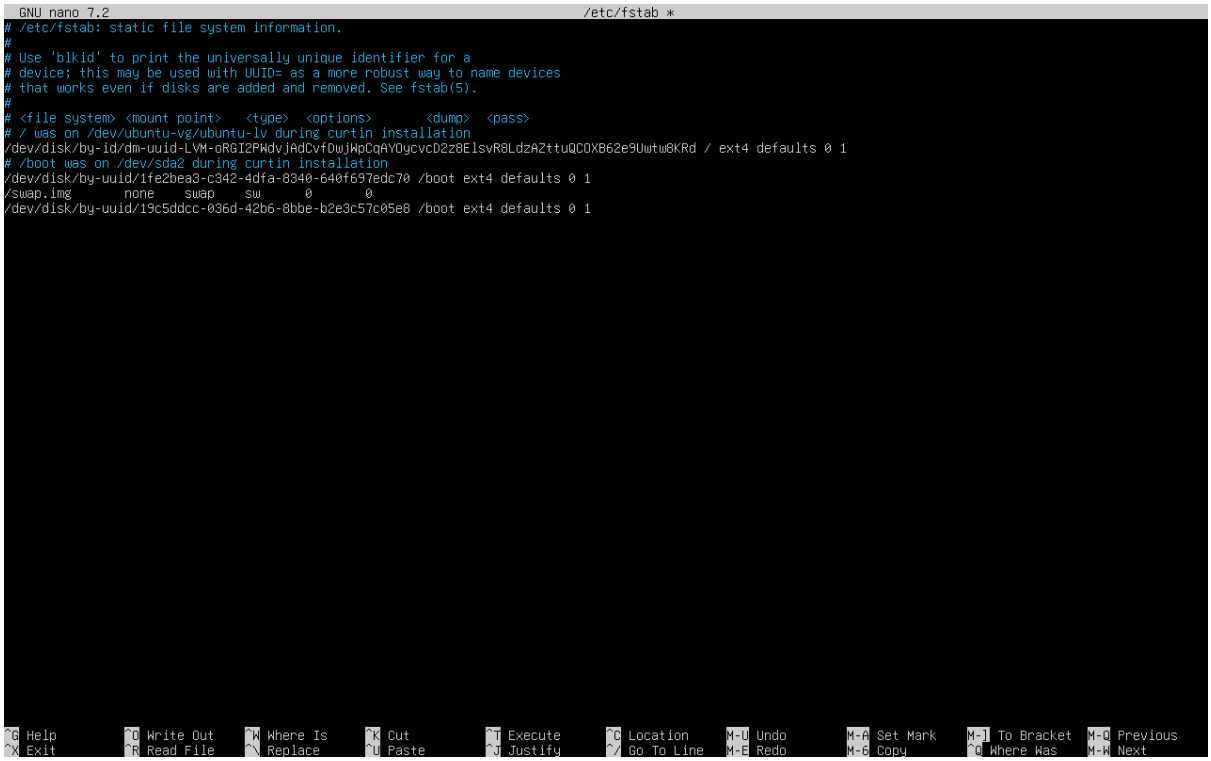


Figur 5: Visar att filsystemet har allokerats rätt med kommandot [fsck]

Efter det behövde jag ändra konfigurationsfilen fstab så att partitionen mountas automatiskt vid boot. Jag tog fram UUID för partitionen och lade till den i konfigfilen enligt instruktioner i själva kofigfilen.



Figur 6: UUID för /dev/sdb1



Figur 7: Konfigurationsfilen fstab

File-types and links

Inode

Inode-numret är ett unikt id för filen/katalogen i filsystemet. Det är filens metadata-arkiv. Inoden håller information om filen, som till exempel: ägare, rättigheter, storlek, pekare till var filens data block finns på disken och mer egenskaper. Låt säga Z är en fil som har inode 5, när man exekverar filen går den till rad 5 i inodetabellen för att läsa metadata och pekare till datasektorer. [3]

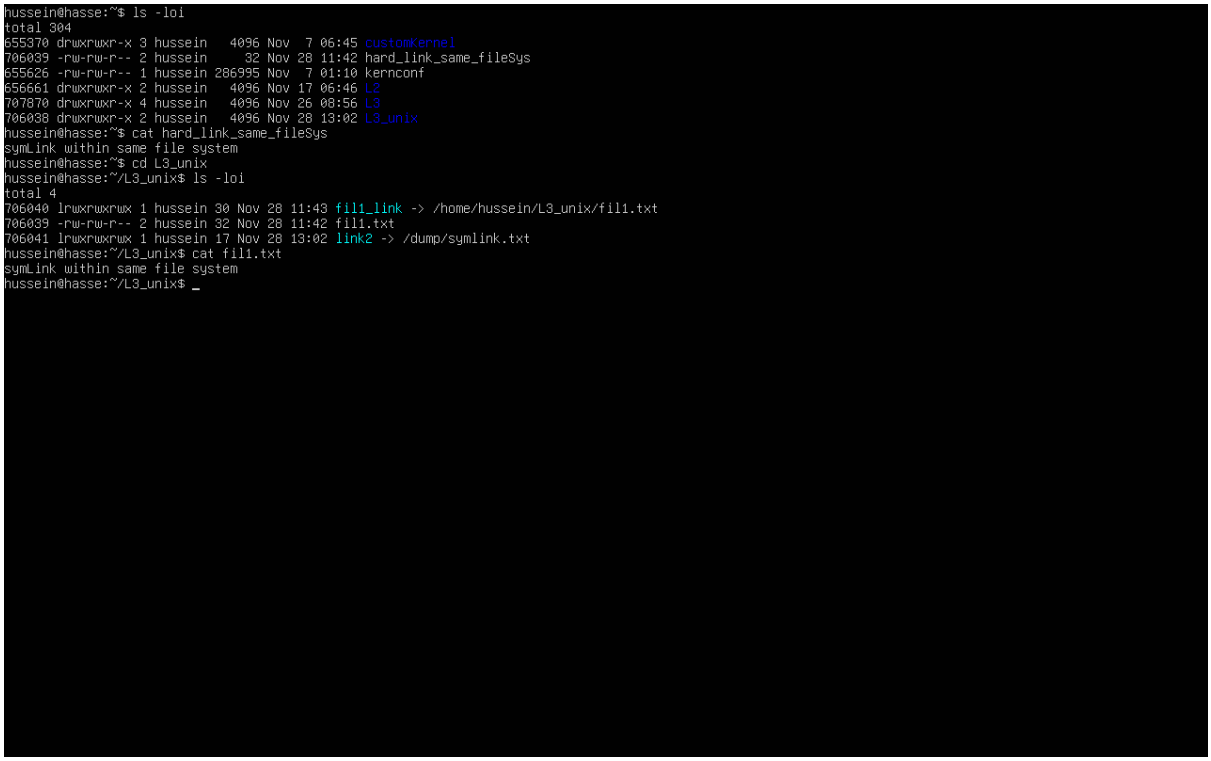
Filtyper

- Special character files - (c)
  System filer som används för att kommunicera med hårdvara. T.ex mus, tangentbord. Dessa filers innehåll genereras dynamiskt av enheten. Det läser data från mikrofon, skriver data till högtalare.
- Directory – (d)
  En vanlig katalog. Går att gå in till den med cd [katalognamn].
- Symbolisk länk – (l)
  Fil som innehåller sökväg till en annan fil eller katalog. Den fungerar som genväg. När systemet försöker öppna eller köra den symboliska länken, följer det sökvägen och arbetar med målfilerna i stället. Den symboliska länken har alltså inget eget innehåll, den hänvisar direkt till en annan fil i filsystemet. Om målfilerna ändras, byts namn eller tas bort påverkas länken direkt. Den kan börja peka på ogiltig plats.
- Hård länk
  Delar inode. Samma filinnehåll. Oavsett om fil A eller fil B som körs, exekveras exakt samma data eftersom de pekar på samma inode.
- Regular file – (-)
  Kan vara textfiler eller skript, .txt eller .py, .log.
  Kan vara binära filer, .exe, .bin, .jpg osv.
- Block device – (b)
  Specialfil som kommunicerar med enhet i block, t.ex hårddiskpartitioner.
- Pipe/FIFO – (p)
  Specialfil som används för kommunikation mellan processer.
- Socket – (s)
  Specialfil som används för nätverkskommunikation.

Denna lista är tagen från boken. [14]

Skillnaden mellan hård länk och symbolisk länk är att hård länk är ett extra filnamn för samma inode som originalet, delar exakt samma metadata och datablock på disken. Tar man bort ena filnamnet, finns filens innehåll kvar så länge minst ett filnamn är kopplat till inoden. Det går att skapa hård länk i samma filsystem, men det går inte att skapa hårdlänk mot ett annat filsystem. Detta för att varje filsystem har en egen inode-tabell, kollar inte på annat filsystems inode-tabell. [14]
Exempel för användande: Om flera program behöver läsa exakt samma konfigurationsfil kan man skapa hårda länkar til den istället för att kopiera filen. Om man uppdaterar konfigurationsfilen som uppdateras den för alla program.

Symbolisk länk är en egen liten fil som innehåller en sökväg i textform. Den fungerar som en genväg. När man försöker öppna symboliska länken, följer systemet sökvägen och öppnar filen den pekar på. Om målet flyttas eller tas bort blir symboliska länken trasig. Symboliska länkar går att peka mot filer/kataloger i andra filsystem. [14]
Exempel för användande: Jag har alla mina bilder i en mapp, kan jag använda symboliska länkar för mina privata program/skripts där jag länkar symboliska länkar mot önskade bilder. Slipper duplicerade bilder.



Figur 8: hård länkning & mjuk länkning

I figur 8 ser man att hårdlänkning ser i samma filsystem, där fil1.txt har inode 706039 och filen som heter hard_link_same_fileSys också har inode 706039.

Här ser man också att mjulänkning kan ske mot annat filsystem där jag har link2 i /home/hussein/L3_unix och den pekar mot /dump/symLink.txt

Users, groups and permissions

Kommandon som använts för att lägga till användare, grupper och addera (append) användare till gruppen:
[sudo adduser <usernam>] – Skapa ny användare

[sudo addgroup <groupname>] – Skapa ny grupp

[sudo usermod -aG <groupname> <username>] – Lägg till användare i grupp

User and groups

- **UID**: Vanligtvis, får root UID 0 på de flesta system. UID är unikt användar-nummer som varje användare tilldelas. Alla filer och processer som ägs av användaren använder detta UID för åtkomstkontroll. UID mellan 1-999 är reserverade för systemanvändare och servicekonton. 1000+ tilldelas till vanliga användare. När ett konto skapa läggs det till i /etc/passwd.
  Ex från min /etc/passwd:
  mickey:x:1002:1002:mickey mouse,,,:/home/mickey:/bin/bash
  Varje rad beskriver en användare där första kolumnen är username, andra är alltid X då det ska föreställa löserord som är krypterade och sparade i /etc/shadow, 1002 är mickeys unika id, andra 1002 är mickeys primär GID. Hans egna grupp där han är ägare och administratör. Sedan hela namnet(mickey mouse). Sedan är det PATH till hemkatalogen och till sist shell.
  Det rekommenderas att inte återanvända UID om man tar bort en användare. [14]
- **GID**: Precis som UID, GID är grupp identifierings nummer. Numret är ett 32-bit heltal. Samma gäller root-gruppen som får GID 0.



Figur 9: Utskrift av /etc/passwd, info om mickey och info om gruppen disney

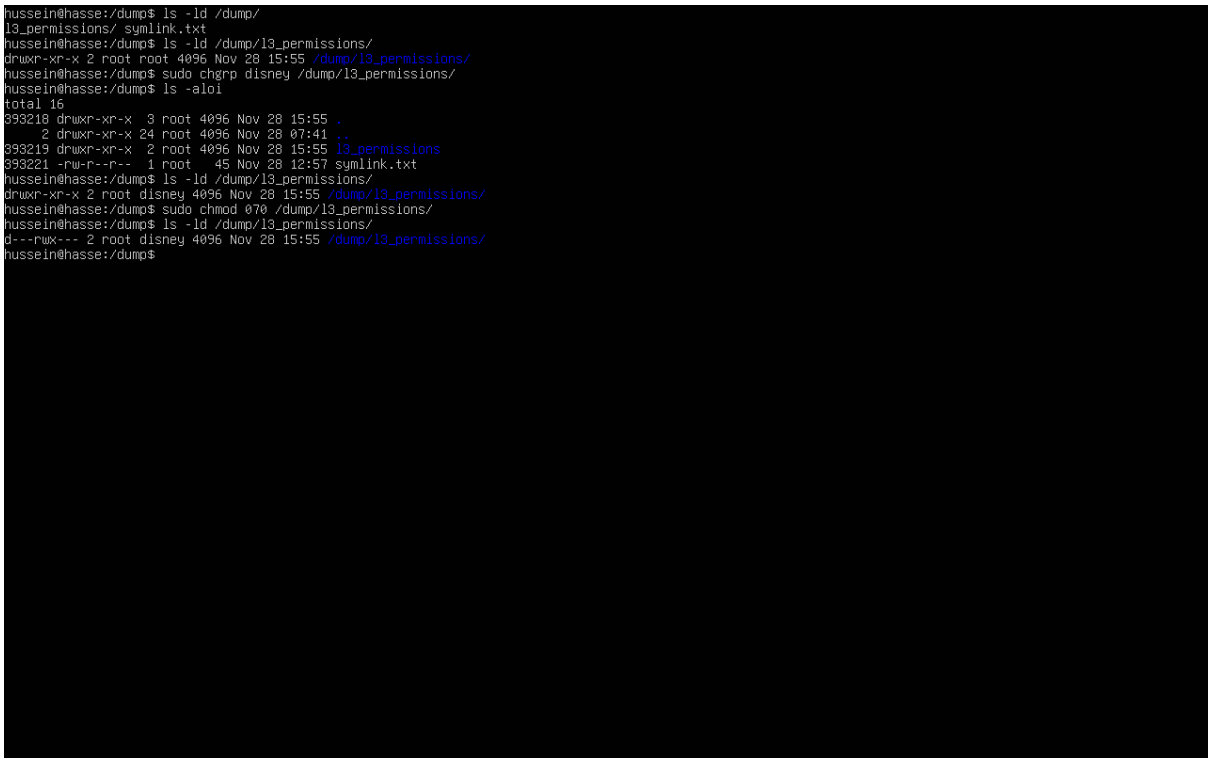
Auto-config new accounts

Jag använde kommandot [sudo mkdir -p /etc/skel/.ssh] där -p skapar katalogen om den inte redan finns.

Sedan använde jag [sudo chmod 700 /etc/skel/.ssh], kommandot säkerställer att endast användare har tillgång till denna katalog.

/etc/skel är en katalog som innehåller standardfiler och inställningar som kopieras till en ny användares hemkatalog när kontot skapas. Alla ny användare får samma grundläggande miljö och konfiguration. När useradd anropas, kopieras allt innehåll från /etc/skel till ny användarens hemkatalog. [5]

Permissions



Figur 10: Bild som visar innan och efter ändrandet av katalogens grupp och rättigheter

-rw -r - -r - - 1 lennart lennart 5496 nov 10 17:40 lab_assgn2 . tex

Detta är en regular file, indikeras med första tecknet(-). Annars är filändelsen .txt alltså en textfil. Filen ägs av Lennart och tillhör Lennarts primära grupp. Subgroup om jag skulle beskriva det med eget ord. Lennart får läsa och skriva till filen, men inte exekvera filen om den skulle vart ett program. Medlemmar i Lennarts grupp och alla andra användare har endast read-only rättigheter för filen. De kan bara läsa den. Filens storlek är 5496 bytes, den faktiska mängden av data filen innehåller. Senaste gången Lennart ändrade filen var 10 november kl 17:40, då ingen annan än Lennart får skriva till filen. Namnet på filen är lab_assgn2.

Backup and file copy

Backup

Tar

Det skriptet jag skrev använder programmet tar. Tar är ett väldigt hjälpsamt verktyg som komprimerar kataloger och filer till en enda arkivfil för att slippa redundans vid till exempel sändning av stora filsystem. Jag lagrar sökvägar till katalogen jag vill

komprimera och till destinationen där jag vill lagra den komprimerade filen i variabler. Jag skapar en variabel för namnet för filen, namnet på filen hämtar hostname och tidpunkt. Jag använder flaggorna cfz. C står för create. Z anropar gzip och f specifierar filnamnet som tilldelas till den färdiga komprimerade filen. [6] [7]



Cpio

Detta skript är nästan identiskt med tar skriptet förutom sista kommandot i filen. I detta skript använder jag [find . -print] för att skriva ut alla filer i nuvarande katalog till standard output (efter cd ”$HOME). Så varje gång man kör skriptet skickas användaren till hemkatalogen. Sedan [cpio -o] som bygger ett akriv av listan. Sedan [-H newc] är ett modern arkivformat. Sist har jag [gzip > <destination>] som komprimerar strömmer med programmet gzip. [8]



Cp

Detta skript skapar en katalog baserat på host samt tidpunk i /dump/home_backup. Sedan kopieras allt innehåll rekursivt inklusivt rättigheter, ägare, grupp, timestamps osv.. från källkatalogen till nya katalogen i /dump, med hjälp av flagga a som är ekvivalent med [cp -rlptgoD]. [cp –help (manualen)]
-(r): Rekursiv kopierar hela trädsturkture, allt hela filsystemet.
-(l): Kopiera länkar.
-(p): Bevara rättigheter.
-(t): Bevara senast ändrad tid.
-(g): Bevara grupp.
-(o): Bevara ägare.
-(D): Bevara enhet- och special filer.

```
  GNU nano 7.2                                  cp_backup_home.sh
#!/bin/bash
BACKUP="$HOME"
DEST="/dump/home_backup"
TIMESTAMP="$(date +'%d_%m_%Y_%H_%M_%S')"
HOSTNAME=$(hostname -s)
BACKUP_DIR="$DEST/${HOSTNAME}_${TIMESTAMP}_cp"
mkdir -p "$BACKUP_DIR"
cp -a "$BACKUP/." "$BACKUP_DIR/"
echo "BACKUP KLAR: $DEST/$ARCHIVE_FILE"

                                [ Read 9 lines ]
^G Help      ^O Write Out   ^W Where Is   ^K Cut         ^T Execute     ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-Q Previous
^X Exit      ^R Read File   ^\ Replace    ^U Paste       ^J Justify     ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was    M-W Next
```

rsync

Detta skript är nästa identiskt med cp-skriptet. Enda skillnaden är att jag använder rsync i sista raden istället för cp. Rsync är att väldigt bra skript byggt med delta-transfer algortim. Algoritmen jämför källa med destination, istället för att överföra alla filer pånytt så överför den endast filer som har förändrats i källa. Jag använde flagga a tillsammans med –progress för att visa progress under överföring. [9]

```
  GNU nano 7.2                                 rsync_backup_home.sh
#!/bin/bash
BACKUP="$HOME"
DEST="/dump/home_backup"
TIMESTAMP="$(date +'%d_%m_%Y_%H_%M_%S')"
HOSTNAME=$(hostname -s)
BACKUP_DIR="$DEST/${HOSTNAME}_${TIMESTAMP}_rsync"
mkdir -p "$BACKUP_DIR"
rsync -a --progress "$BACKUP/." "$BACKUP_DIR/"
echo "BACKUP KLAR: $BACKUP_DIR"

                                [ Read 9 lines ]
^G Help      ^O Write Out   ^W Where Is   ^K Cut         ^T Execute     ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-Q Previous
^X Exit      ^R Read File   ^\ Replace    ^U Paste       ^J Justify     ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was    M-W Next
```

dd

```
hussein@hasse:~$ sudo dd if=/etc/passwd of=/dump/passwd_upperC conv=ucase
[sudo] password for hussein:
3+1 records in
3+1 records out
1902 bytes (1.9 kB, 1.9 KiB) copied, 7.717e-05 s, 24.6 MB/s
hussein@hasse:~$ cat /dump/
home_backup/    l3_permissions/ passwd_upperC   symlink.txt
hussein@hasse:~$ cat /dump/passwd_upperC
ROOT:X:0:0:ROOT:/ROOT:/BIN/BASH
DAEMON:X:1:1:DAEMON:/USR/SBIN:/USR/SBIN/NOLOGIN
BIN:X:2:2:BIN:/BIN:/USR/SBIN/NOLOGIN
SYS:X:3:3:SYS:/DEV:/USR/SBIN/NOLOGIN
SYNC:X:4:65534:SYNC:/BIN:/BIN/SYNC
GAMES:X:5:60:GAMES:/USR/GAMES:/USR/SBIN/NOLOGIN
MAN:X:6:12:MAN:/VAR/CACHE/MAN:/USR/SBIN/NOLOGIN
LP:X:7:7:LP:/VAR/SPOOL/LPD:/USR/SBIN/NOLOGIN
MAIL:X:8:8:MAIL:/VAR/MAIL:/USR/SBIN/NOLOGIN
NEWS:X:9:9:NEWS:/VAR/SPOOL/NEWS:/USR/SBIN/NOLOGIN
UUCP:X:10:10:UUCP:/VAR/SPOOL/UUCP:/USR/SBIN/NOLOGIN
PROXY:X:13:13:PROXY:/BIN:/USR/SBIN/NOLOGIN
WWW-DATA:X:33:33:WWW-DATA:/VAR/WWW:/USR/SBIN/NOLOGIN
BACKUP:X:34:34:BACKUP:/VAR/BACKUPS:/USR/SBIN/NOLOGIN
LIST:X:38:38:MAILING LIST MANAGER:/VAR/LIST:/USR/SBIN/NOLOGIN
IRC:X:39:39:IRCD:/RUN/IRCD:/USR/SBIN/NOLOGIN
_APT:X:42:65534::/NONEXISTENT:/USR/SBIN/NOLOGIN
NOBODY:X:65534:65534:NOBODY:/NONEXISTENT:/USR/SBIN/NOLOGIN
SYSTEMD-NETWORK:X:998:998:SYSTEMD NETWORK MANAGEMENT:/:/USR/SBIN/NOLOGIN
SYSTEMD-TIMESYNC:X:997:997:SYSTEMD TIME SYNCHRONIZATION:/:/USR/SBIN/NOLOGIN
DHCPCD:X:100:65534:DHCP CLIENT DAEMON,,,:/USR/LIB/DHCPCD:/BIN/FALSE
MESSAGEBUS:X:101:102::/NONEXISTENT:/USR/SBIN/NOLOGIN
SYSTEMD-RESOLVE:X:992:992:SYSTEMD RESOLVER:/:/USR/SBIN/NOLOGIN
POLLINATE:X:102:1::/VAR/CACHE/POLLINATE:/BIN/FALSE
POLKITD:X:991:991:USER FOR POLKITD:/:/USR/SBIN/NOLOGIN
SYSLOG:X:103:104::/NONEXISTENT:/USR/SBIN/NOLOGIN
UUIDD:X:104:105::/RUN/UUIDD:/USR/SBIN/NOLOGIN
TCPDUMP:X:105:107::/NONEXISTENT:/USR/SBIN/NOLOGIN
TSS:X:106:108:TPM SOFTWARE STACK,,,:/VAR/LIB/TPM:/BIN/FALSE
LANDSCAPE:X:107:109::/VAR/LIB/LANDSCAPE:/USR/SBIN/NOLOGIN
FWUPD-REFRESH:X:989:989:FIRMWARE UPDATE DAEMON:/VAR/LIB/FWUPD:/USR/SBIN/NOLOGIN
USBMUX:X:108:46:USBMUX DAEMON,,,:/VAR/LIB/USBMUX:/USR/SBIN/NOLOGIN
SSHD:X:109:65534::/RUN/SSHD:/USR/SBIN/NOLOGIN
HUSSEIN:X:1000:1000:HUSSEIN:/HOME/HUSSEIN:/BIN/BASH
DONALD:X:1001:1001:DONALD TRUMP,,,:/HOME/DONALD:/BIN/BASH
MICKEY:X:1002:1002:MICKEY MOUSE,,,:/HOME/MICKEY:/BIN/BASH
hussein@hasse:~$ _
```

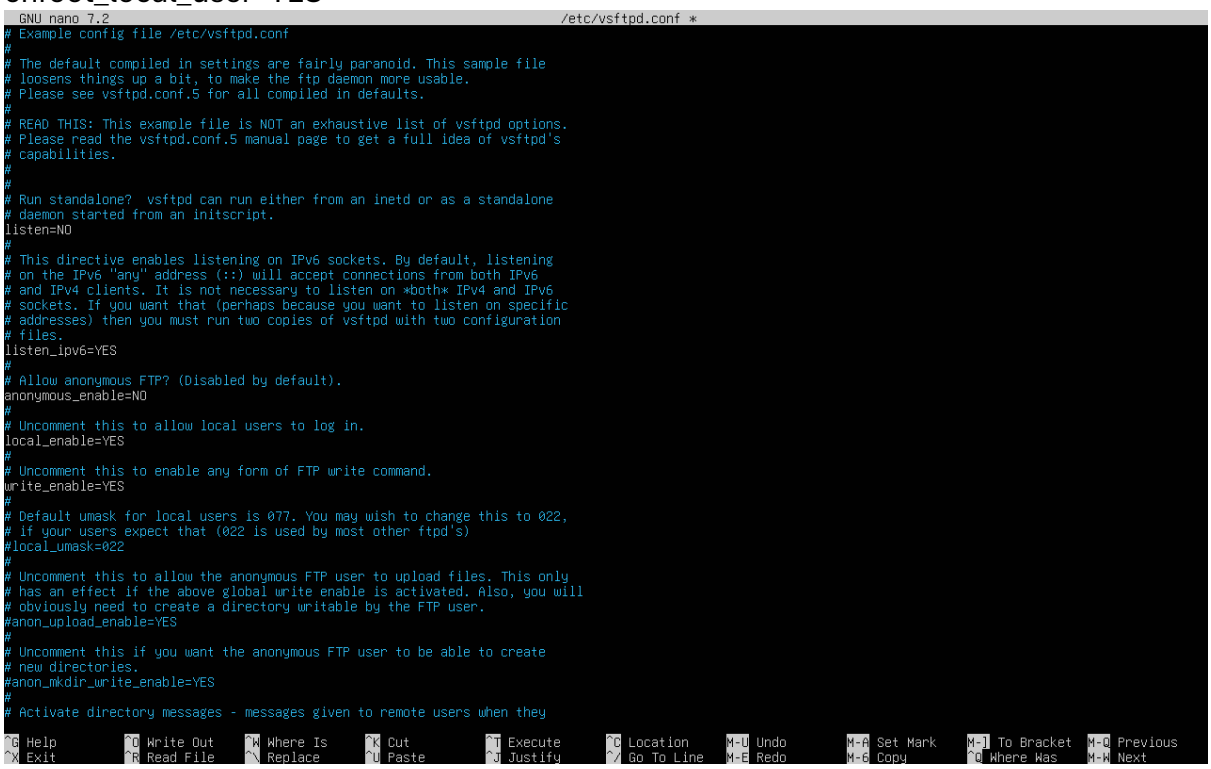
Figur 11: Kopierar allt innehåll i /etc/passwd till en ny fil och omvandlar allt till stora bokstäver

Sharing files

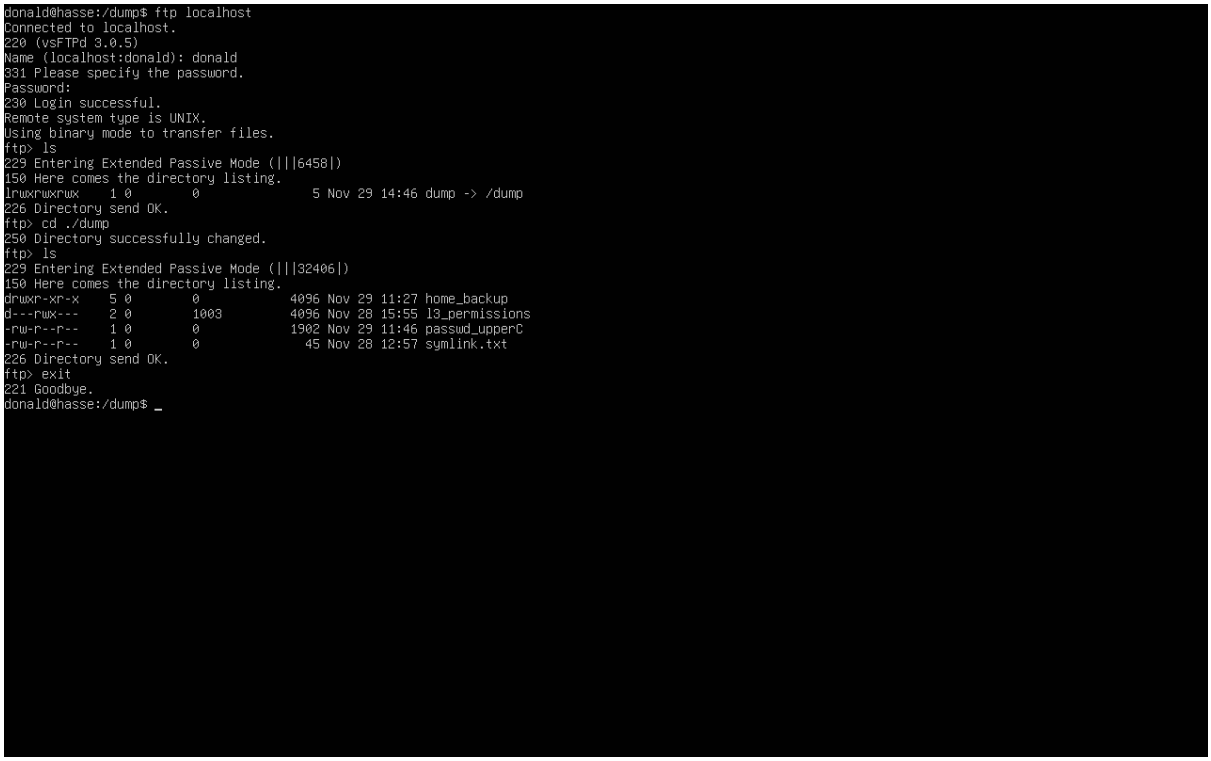
FTP

Jag installerade FTP-servern vsftpd med kommandot [sudo apt install vsftpd]. Därefter konfigurerade jag filbehörigheterna så att användarna endast kan nå sina egna hemkataloger och endast medlemmar i disney kan komma åt /dump. [12]
Jag avkommenterade följade inställningar i konfigfilen:

local_enable=YES
write_enable=YES
ascii_upload_enable=YES
acii_download_enable=YES
chroot_local_user=YES



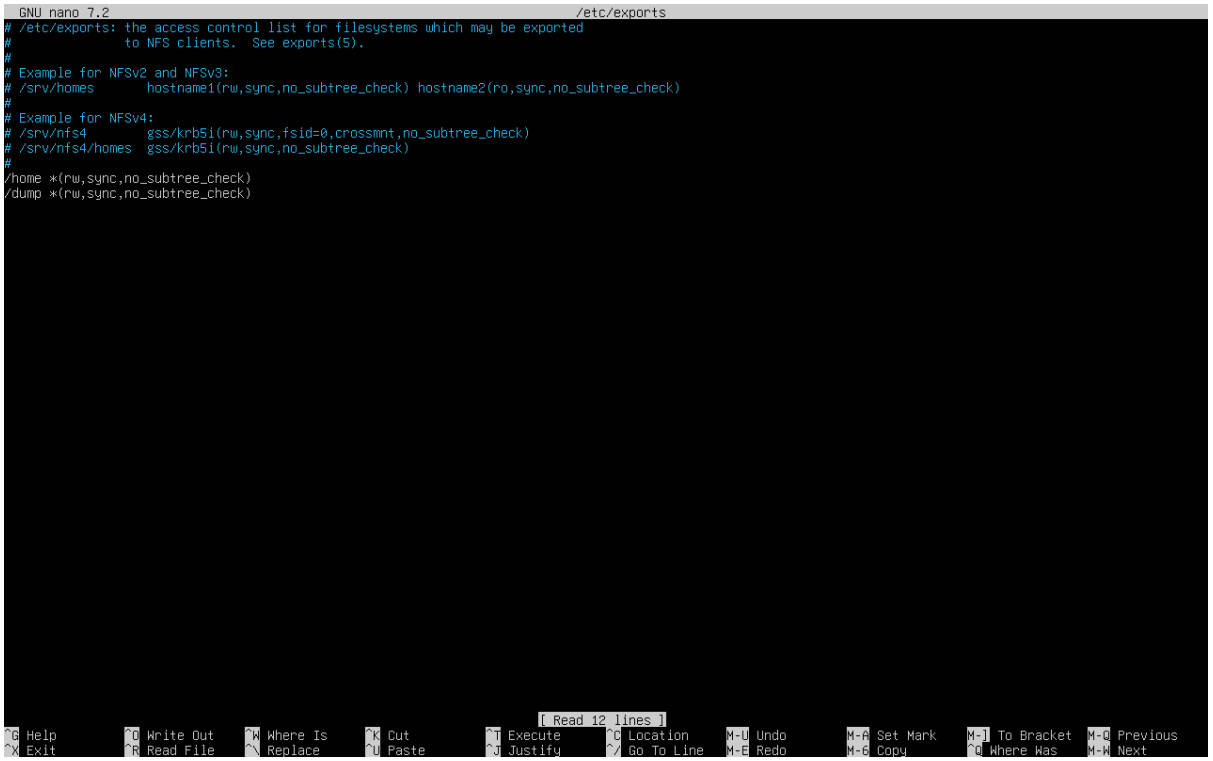
Figur 12: FTP konfigfil



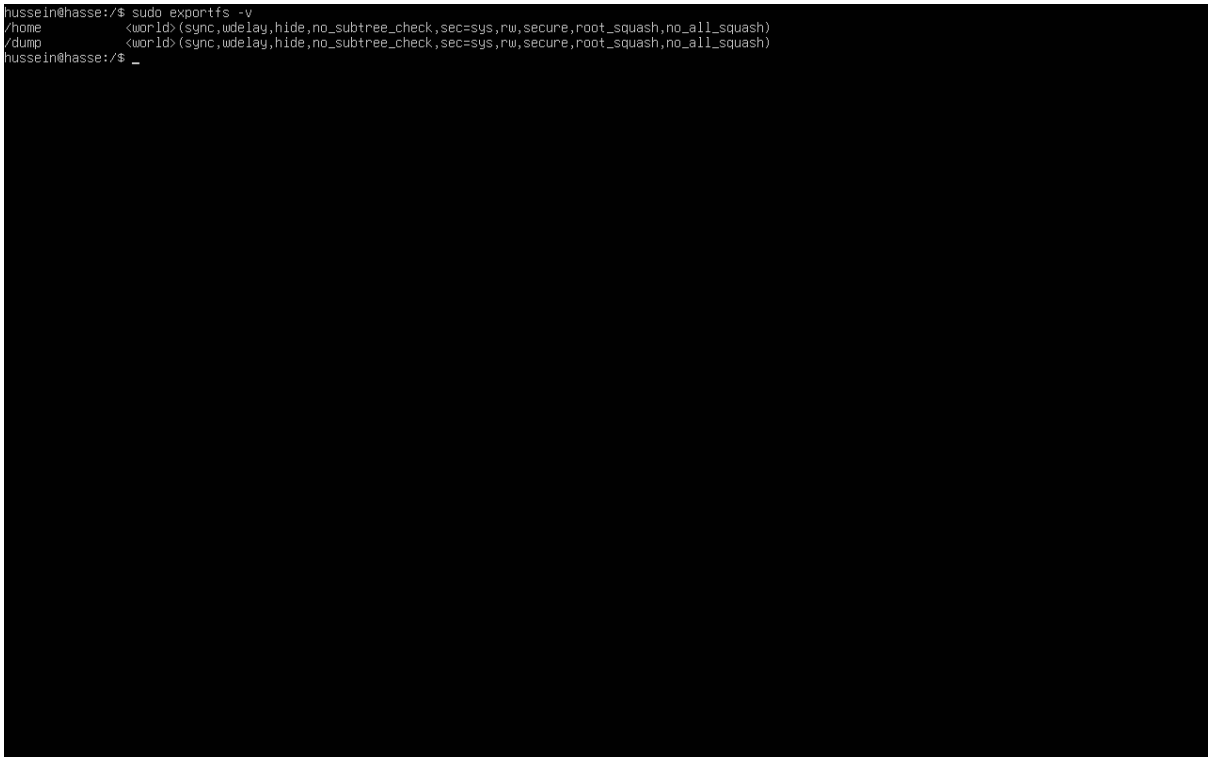
Figur 13: Navigering till dump via donalds hemkatalog

NFS

NFS-servern installerades med [sudo apt install nfs-kernel-server]. För att dela kataloger över nätverket redigerades /etc/exports. Den filen används av NFS-servern för att specifiera vilka kataloger som delas ut över nätverket och vilka röttigeter. [10]



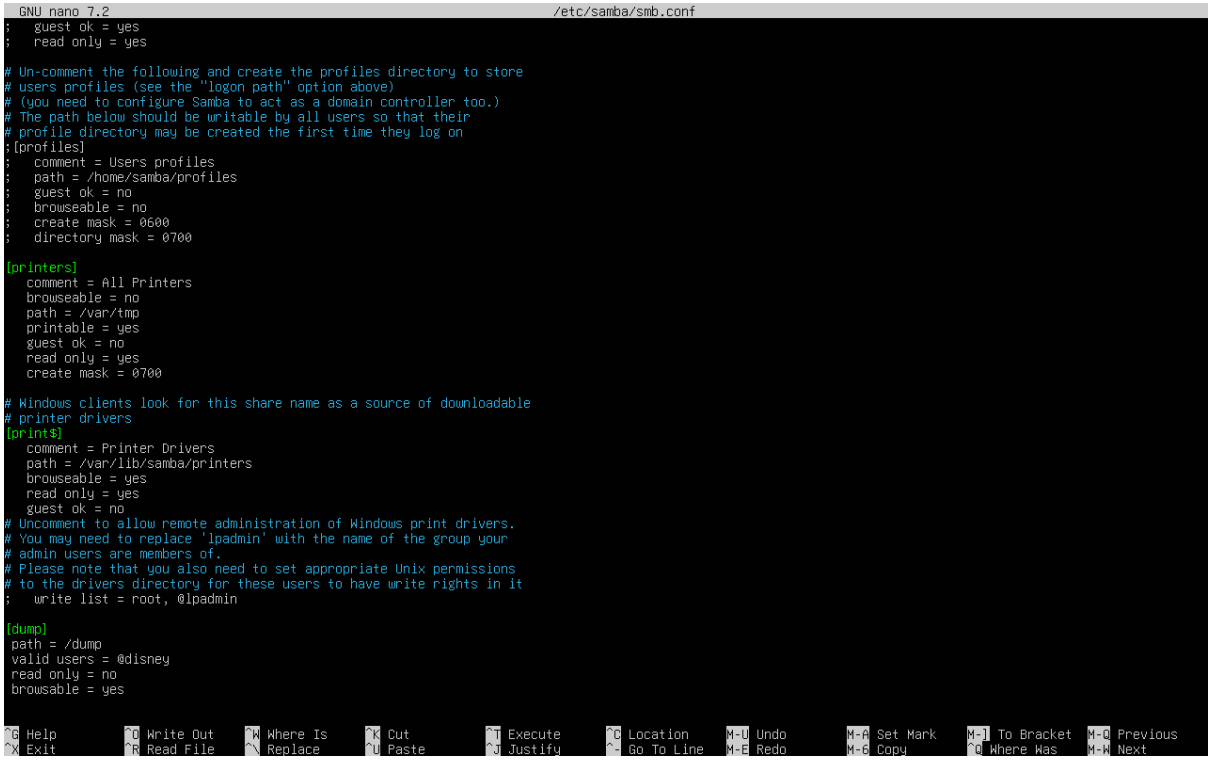
Figur 14: /etc/exports



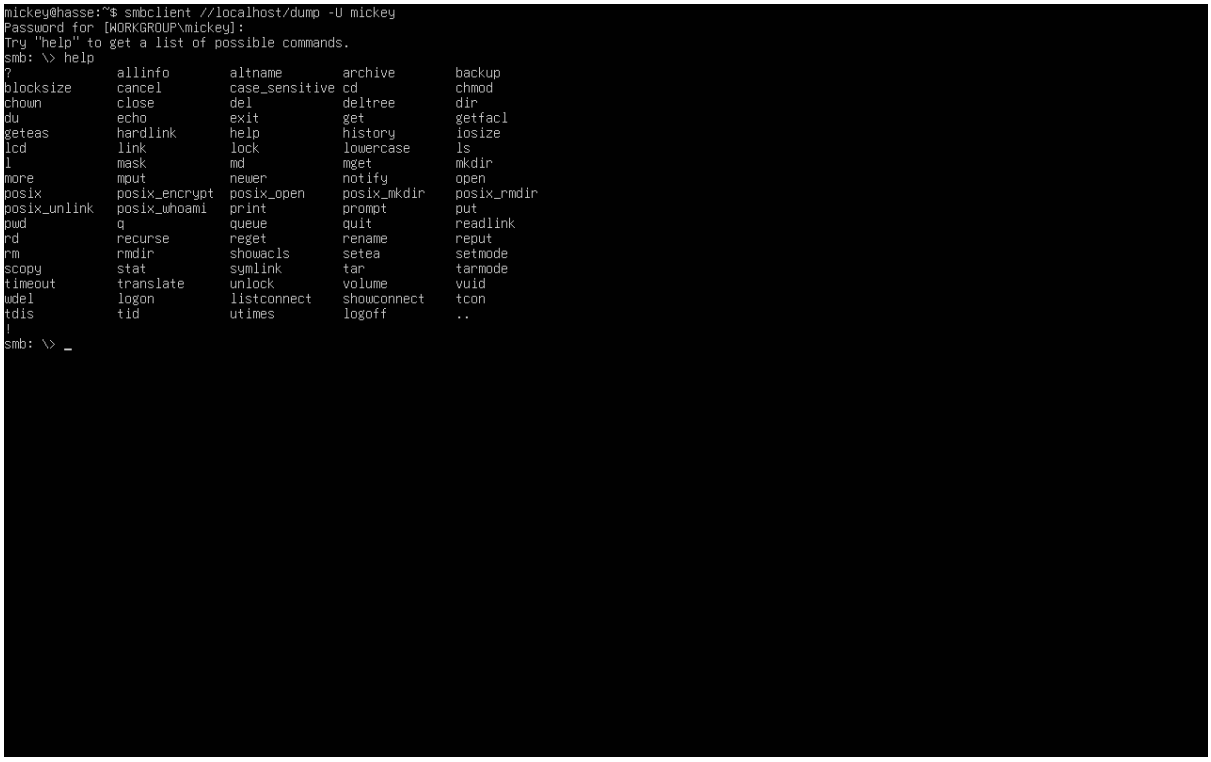
Figur 15: Kataloger som delas ut

Samba

Samba installerade med [sudo apt install samba]. /dump användes som den delade katalogen. Samba-användare skapades med [smbpasswd] för de befintliga användarna. I /etc/samba/smb.conf lades ett block för /dump, där disnet gruppen har åtkomst. Samba startades om med [sudo systemctl restart smbd]. [11]



Figur 16: Sambas konfigfil



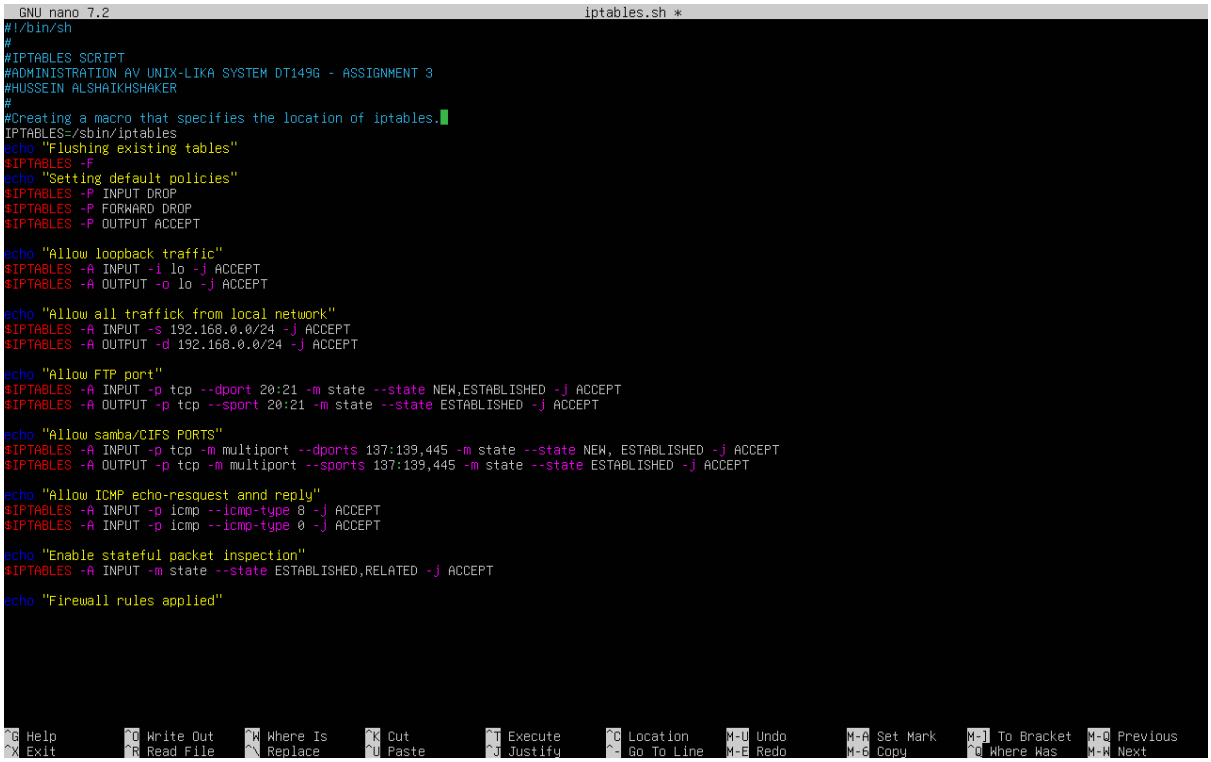
Figur 17: Inne i samba client via mickeys konto

Skillnader

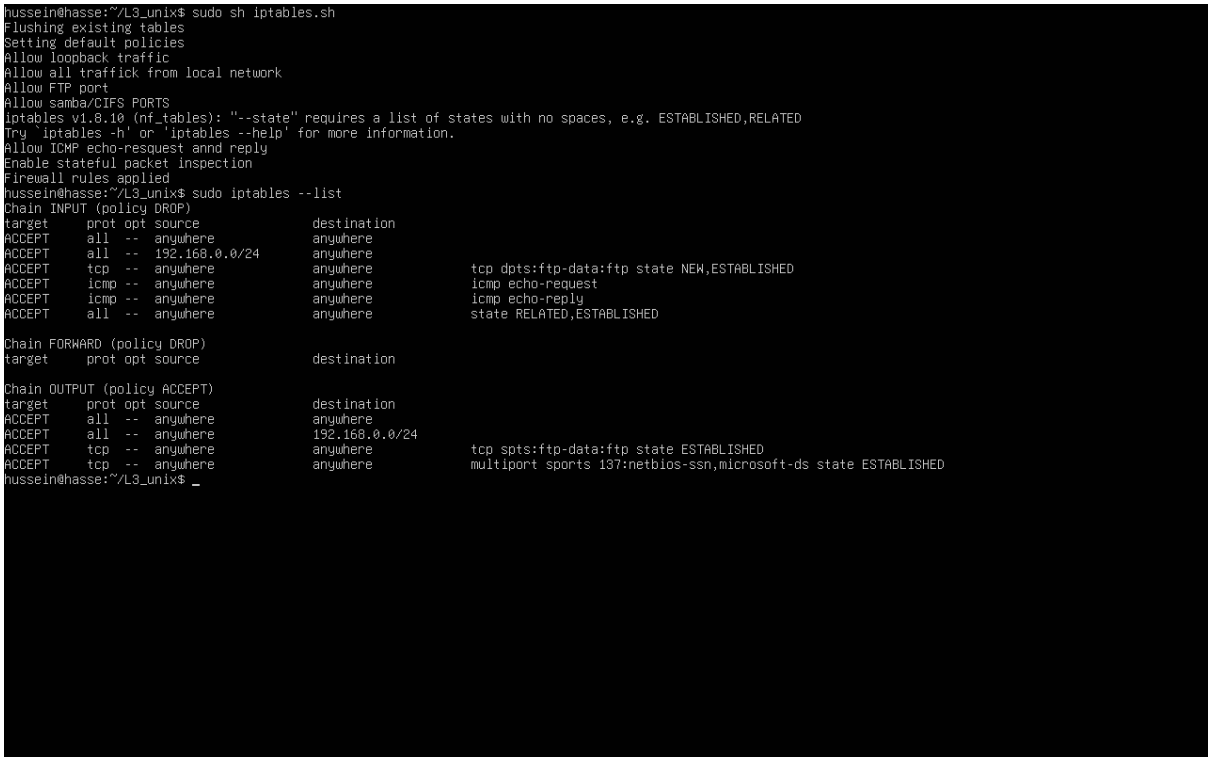
FTP använder systemets unix-användare och behörigheter för att bestämma åtkomst. Hemmamappar kan göras privata. Mappar som ligger i ett annat filsystem kan nås med mjuka länkar.
NFS exporterar kataloger över nätverket, men unix-behörigheter på servern gäller fortfarande. Man styr vilka kataloger som delas via /etc/exports och vilka klienter som kan ansluta.
Samba använder både unix-behörigheter och samba-specifika användare. Man kan begränsa åtkomst per användare eller grupp i sambas konfigfil.

Firewall

När stateful packet inspection är aktiverat följer brandväggen inte bara enstaka paket utan hela nätverksförbindelser. Den kommer ihåg vilka anslutningar som redan är öppnade av systemet och kan avgöra om inkommande paket hör till en sådan anslutning. Trafik som redan är etablerad släpps igenom, medan oväntade anslutningar blockeras. [13]



Figur 18: custom firewall skript



Figur 19: Custom firewall aktiverad

Referenser

(u.d.). Hämtat från https://man7.org/linux/man-pages/man1/

(u.d.). Hämtat från https://man7.org/linux/man-pages/man2/

(u.d.). Hämtat från https://en.wikipedia.org/wiki/Inode

(u.d.). Hämtat från http://techtarget.com/searchdatacenter/definition/runlevel

(u.d.). Hämtat från https://www.linfo.org/etc_skel.html

(u.d.). Hämtat från https://www.w3schools.com/bash/bash_tar.php

(u.d.). Hämtat från https://www.reddit.com/r/compsci/comments/jh2wf1/how_does_compression_work_really/

(u.d.). Hämtat från https://www.ibm.com/docs/en/aix/7.2.0?topic=c-cpio-command

(u.d.). Hämtat från https://github.com/RsyncProject/rsync

(u.d.). Hämtat från https://medium.com/@manankathrecha/how-to-install-and-configure-an-nfs-server-on-ubuntu-59472f644403

(u.d.). Hämtat från https://ubuntu.com/tutorials/install-and-configure-samba#1-overview

(u.d.). Hämtat från https://documentation.ubuntu.com/server/how-to/networking/ftp/

(u.d.). Hämtat från https://linux.die.net/man/8/iptables

Nemeth, E. (2017). *Unix and Linux System Administration Handbook*. Addison-Wesley.